

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Studiul și implementarea soluțiilor cloud scalabile
pentru aplicații web**

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare și tehnologia informației*
programul de studii *Ingineria informației***

Conducător științific
Ș.L.Dr.Ing. George Valentin STOICA

Absolvent
Ionescu Maria Cătălina

Anul 2025

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2025.

Absolvent: Ionescu Maria Cătălina

.....

Cuprins

Lista figurilor	iii
Lista tabelelor	iv
Lista acronimelor	v
1. Introducere	1
1.1. Prezentarea proiectului	1
1.2. Scopul proiectului	1
1.3. Structura lucrării	2
2. Aspectele teoretice ale proiectului	3
2.1. Aplicația web	3
2.2. Protocolul HTTP	4
2.2.1. Componentele sistemelor bazate pe HTTP	4
2.2.1.1. Clientul: user-agent-ul	4
2.2.1.2. Server-ul web	4
2.2.1.3. Componente URL	5
2.2.2. Conexiunea HTTP	7
2.2.2.1. Cum funcționează o conexiune HTTP	7
2.2.3. Mesaje HTTP	9
2.2.3.1. Structura mesajelor HTTP	9
2.2.3.2. Cereri HTTP	10
2.2.3.3. Răspunsuri HTTP	14
2.3. Flask Python	16
3. Structura aplicației	18
4. Noțiuni teoretice privind aplicațiile web și scalabilitatea	20
4.1. Alte recomandări	20
5. Basmе	21
5.1. Punguța cu doi bani	21
5.2. Tinerețe fără bătrânețe și viață fără de moarte	21
5.3. Cinci Pâini	23
Bibliografie	26
Anexa A. Rezolvare ”Cinci Pâini”	28
Anexa B. Alte povești și basme	29

Lista figurilor

2.1. Sintaxa user-agent-ului	4
2.2. Componentele URL-ului	5
2.3. Tipul de protocol URL	5
2.4. Numele resursei și portul	6
2.5. Calea către resursă	6
2.6. Parametrii	6
2.7. Ancoră URL	7
2.8. Diagrama modului de funcționare a server-ului web	7
2.9. Diagrama conexiunii HTTP	8
2.10. Cerere HTTP	8
2.11. Răspuns HTTP	8
2.12. Răspuns HTTP	9
2.13. Cerere HTTP	10
2.14. Cerere HTTP	10
2.15. Formatul unei cereri HTTP	11
2.16. Forma de origine	11
2.17. Forma absolută	12
2.18. Forma de autoritate	12
2.19. Forma asterisk	12
2.20. Header request	13
2.21. Date din corp sub formă de key=values	13
2.22. Date din corp împărțite în mai multe părți	14
2.23. Răspuns HTTP	14
2.24. Alcătuirea răspunsului HTTP	15
2.25. Status line - alcătuire	15
2.26. Exemplu de response și representation header	16
5.1. Cucoșu găsește o pungă cu doi galbeni.	22

Lista tabelelor

B.1. Alte basme.	29
--------------------------	----

Lista acronimelor

HTML = HyperText Markup Language

HTTP = Hypertext Transfer Protocol

IoT = Internet of Things

VM1 = Mașină virtuală 1

VM2 = Mașină virtuală 2

Capitolul 1

Introducere

1.1 Prezentarea proiectului

Cerințele privind performanța aplicațiilor web sunt în continuă creștere datorită gamei extinse de dispozitive inteligente și IoT (Internet of Things) folosite de către utilizatori pentru a-și îmbunătăți viața de zi cu zi. Numărul mare de dispozitive care accesează în mod concomitent un site web duce la un volum copleșitor de cereri către servere pentru a accesa informația cerută, ceea ce poate cauza o experiență întârziată sau chiar imposibilă a utilizatorului. Datorită acestor circumstanțe, traficului de date crește, iar aplicațiile web trebuie să facă față la un flux considerabil de informații. La momentul actual, o soluție pentru optimizarea aplicațiilor web expuse unui trafic mare de date, dar și pentru a spori satisfacția utilizatorilor este folosirea soluțiilor scalabile a resurselor software și hardware.

Scalabilitatea [1] reprezintă capacitatea unui sistem de a-și folosi în mod adaptiv și utiliza eficient resursele în scopul creșterii performanței și reducerii costurilor, în funcție de cerințele de procesare. Aceasta se clasifică după tipul de resurse folosite în:

1. Scalabilitatea resurselor software [2]: se referă la abilitatea unei aplicații de a-și adapta sau menține performanța atunci când este o creștere sau o scădere bruscă a volumului de utilizatori, date sau operațiuni.
2. Scalabilitatea resurselor hardware [3]: reprezintă capacitatea unui sistem de a-și mări capacitatea sau de a-și crește performanța utilizând noi resurse hardware.

De asemenea, folosirea în paralel a unei bazei de date distribuită reprezintă o soluție eficientă pentru asigurarea redundanței datelor și a performanței optime într-o aplicație complexă.

O bază de date distribuită [4] este o bază de date controlată de un sistem de gestiune a bazelor de date (Data Base Management System), în care datele sunt distribuite la mai multe calculatoare interconectate.

1.2 Scopul proiectului

Proiectul „**Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web**” își propune studierea, implementarea și testarea soluțiilor scalabile software pentru o aplicație web, utilizând framework-ul Flask Python (pentru dezvoltarea aplicației web), MySQL (sistem de gestionare a bazelor de date relaționale) și Nginx (loadbalancer și server web de tip reverse proxy).

Scopul acestui proiect este a urmări și analiza capabilitatea aplicației de a răspunde la un număr mare de cereri simultane, îmbunătățirea performanței și reducerea redundanței prin implementarea unei soluții scalabile de tip software care include un load balancer și o bază de date distribuită master-slave.

Aplicația, o platformă de tip rezervări, a fost dezvoltată într-un mediu de lucru alcătuit dintr-o stație de lucru, alături de un mediu de testare, constituit de cele două mașini virtuale instalate pe stația de lucru. Împreună, cele trei mașini constituie o rețea IP/LAN privată. De

asemenea, la nivel arhitectural s-a implementat o bază de date distribuită de tip master-slave pentru a optimiza operațiile de citire-scriere.

Obiectivele specifice acestei lucrări sunt:

- Dezvoltarea aplicației web folosind framework-ul Flask Python care include rute HTTP pentru funcționalități de bază precum ar fi: autentificarea (login, register), managementul utilizatorilor (profile), administrarea rezervărilor (reserve) și pagini administrative (admin).
- Implementarea arhitecturii scalabile prin intermediul sistemului de load balancing folosind Nginx. Nginx are rolul de a distribui cererile trimise între cele două servere disponibile, aplicația Flask și VM2, rezultând în creșterea performanței și puterii de procesare a aplicației web.
- Optimizarea performanței bazei de date cu un server master pe stația de lucru și un server slave pe VM1. Cele două servere îmbunătățesc operațiile de citire și scriere și asigură redundanța datelor prin atribuirea operațiilor de citire serverul-ui master, iar operațiile de scriere serveru-lui slave.

Semnificația practică a acestei lucrări constă în evidențierea rezultatelor obținute în urma aplicării soluțiilor scalabile implementate pentru a facilita o performanță crescută la un cost scăzut pentru a gestiona eficient cererile utilizatorilor. Rezultatele oferă o perspectivă clară asupra comportamentului unei aplicație web atunci când este supusă la un volum semnificativ de cereri, dar și asupra faptului cum pot fi îmbunătățite performanța și costurile de operare .

1.3 Structura lucrării

Capitolele următoare vor prezenta principalele concepte și soluții abordate în cadrul acestei lucrări, cum ar fi: aspectele teoretice ale proiectului, tehnologiile folosite în implementare și structura aplicației. Acestea sunt urmate de descrierea arhitecturii, în care sunt evidențiate diferitele module ce constituie aplicația și interacțiunea dintre ele la nivel scalabil. Capitolele vor cuprinde și exemplificări cu fragmente de cod care au un rol crucial în aplicarea fenomenului de scalabilitate asupra aplicației web. De asemenea, lucrarea abordează explicații amănunțite despre modul de funcționare și utilizare a interfeței grafice a aplicației cu ajutorul capturilor de ecran.

Capitolul 2

Aspectele teoretice ale proiectului

În mod ideal, scalabilitatea presupune gestionarea optimă a resurselor hardware și software, astfel încât atunci când aplicația este expusă asupra unui număr mare de cereri, ea va trebui să le gestioneze eficient, fără a compromite viteza de răspuns sau resursele disponibile. Scalabilitatea software [5] reprezintă implementarea unor soluțiilor de tip software care asigură funcționarea eficientă a aplicației la un volum de utilizatori brusc variabil. Aceasta folosită concomitent cu scalabilitatea hardware, ce are rol în extinderea resurselor fizice ale unui sistem, poate aduce la rezultate impresionante în gestionarea traficului și a numărului de cereri semnificativ pentru a oferi utilizatorilor o experiență cât mai fluidă.

În practică, oricât de mult am încerca să avem o viteză de răspuns cât mai optimă, aceasta implică un cost foarte mare al resurselor software, hardware dar și a energiei electrice folosite. Un volum limitat de resurse hardware și software implică, de asemenea, o eficiență de gestionare a cererilor dintr-o aplicației mai scăzută. Astfel, este necesară o arhitectură software și hardware bine definită pentru a reuși în atingerea unor rezultate promițătoare cu un număr minim de resurse folosite.

Această lucrare urmărește să evidențieze importanța soluțiilor scalabile software implementate în cadrul unei aplicații de tip web care are rol în optimizarea și fluidizarea volumului de trafic de date rezultat în urma cererilor transmise de utilizatori către server.

2.1 Aplicația web

O aplicație web [6] este o aplicație de tip software accesată prin intermediul unui browser web care poate rula pe un server local sau cloud. Aceasta poate fi accesată de pe orice tip de dispozitiv conectat la internet cum ar fi calculatoarele, telefoanele, consolele și este implementată prin intermediul unor limbaje de programare web ca HTML, CSS și JavaScript. Ceea ce face diferită o aplicație web față de cele mobile sau desktop este faptul că poate să ruleze pe unul sau pe mai multe servere și este nevoie doar de un browser web pentru a o accesa.

Principalele caracteristici și funcționalități ale unei aplicații web sunt:

- **Scalabilitatea:** datorită faptului că aplicațiile web rulează pe un server există posibilitatea de a implementa soluții scalabile care să îmbunătățească și optimizeze performanța pentru a răspunde la cerințele utilizatorilor.
- **Accesibilitatea:** oricine are un dispozitiv cu conexiune la internet poate să acceseze aplicațiile web, indiferent de locație sau specificațiile hardware și software pe care le are dispozitivul folosit.
- **Integrarea cu alte servicii și aplicații:** aplicații web fiind versatile, programatorii au posibilitatea de a rafina experiența utilizatorilor cu ajutorul serviciilor și aplicațiilor web suplimentare concepute pentru a personaliza experiența fiecărei persoane.
- **Actualizarea eficientă:** odată ce s-a confirmat de către echipa de testare faptul că noua versiune a aplicației web a trecut toate testele și îndeplinește toate cerințele, aceasta

poate fi lansată către public fără a fi necesară intervenția din partea lui, actualizarea fiind automată pe server.

2.2 Protocolul HTTP

În contextul vremurilor actuale, aplicațiile web au devenit unul dintre cele mai rapide, populare, dar și accesibile metode de a accesa informații și servicii de către utilizatori, doar la un click distanță. La baza acestora se află o arhitectură de tip client-server, unde interacțiunile sunt realizate prin intermediul protocoalelor de comunicație.

Protocolul HTTP (Hypertext Transfer Protocol) [7] este protocolul de bază al oricărui schimb de date pe internet care se ocupă cu preluarea resurselor, cum ar fi documentele HTML (HyperText Markup Language), fișiere audio, video ș.a.m.d. Fiind un protocol de tip client-server, cererile sunt inițializate de către client (browser-ul web). Comunicarea dintre clienți și servere are loc prin schimbul de mesaje individuale, unde mesajele trimise de client poartă numele de cereri, iar mesajele trimise de către server - răspunsuri.

2.2.1 Componentele sistemelor bazate pe HTTP

HTTP este un protocol de nivel de aplicație, fără reținerea stării (serverul nu reține date despre sesiunea activ) ce urmează un model de tip client-server în care un client începe o conexiune pentru a face o cerere, așteaptă până primește un răspuns de la server, iar serverul în cele din urmă face schimb de informații cu clientul.

2.2.1.1 Clientul: user-agent-ul

User-agent-ul [8] este un header de tip string folosit într-o cerere HTTP pentru a trimite informații despre aplicație, sistemul de operare, furnizorul și/sau versiunea folosită către server.

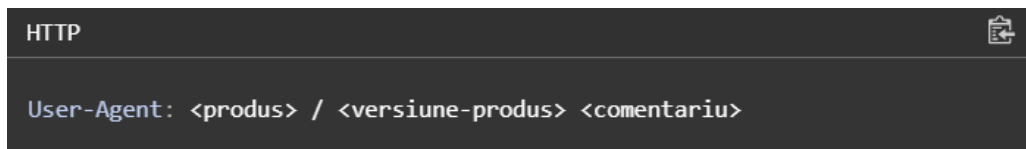


Figura 2.1: Sintaxa user-agent-ului

Ca să afișeze o pagină web, în prima fază browser-ul trimite o cerere inițială pentru a prelua documentul HTML care reprezintă pagina. În cea de-a doua fază, acesta analizează fișierul primit și trimite cereri suplimentare pentru a obține informații despre stilizarea paginii (CSS), eventuale scripturi ce trebuiesc executate și sub-resursele aflate în pagină (imagini sau videoclipuri). În fază finală a procesului, browser-ul combină toate resursele pentru a prezenta documentul complet.

2.2.1.2 Server-ul web

Server-ul web [9] se ocupă cu livrarea resurselor cerute de către client. Din punct de vedere al structurii hardware, poate fi un calculator sau o colecție de servere conectate în aceeași rețea care se ocupă cu schimbul de date stocate (de exemplu documente HTML, imaginii, fișiere JavaScript) cu alte dispozitive.

Pe de altă parte, din punct de vedere al structurii software, un server web are rolul de a înțelege adresele URL și protocolul HTTP.

2.2.1.3 Componente URL

URL (Uniform Resource Locators) [10] reprezintă adresa unei resurse unice de pe internet și unul dintre mecanismele folosite de browsere pentru a prelua resurse cum ar fi pagini HTML, documente, imagini ș.a.m.d. Orice URL introdus în bara de adrese duce la o solicitare directă către un server web pentru a accesa resursa specifică, iar în funcție de răspunsul serverul-ui resursa solicitată va fi afișată. Conform figurii 2.8, un URL este compus din diferite componente precum ar fi tipul de protocol URL, numelei resursei, portul, calea către resursă, parametrii și fragmentul de identificare internă.

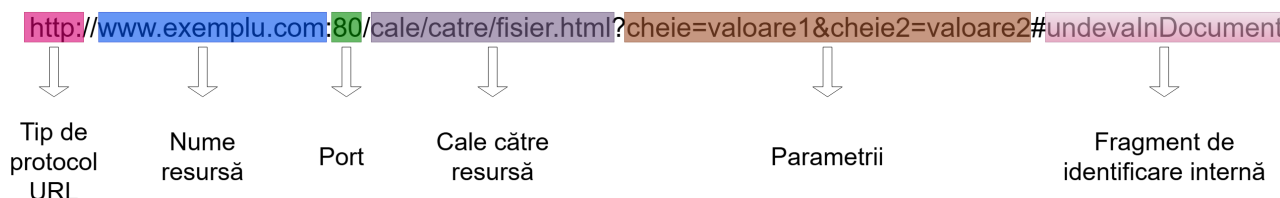


Figura 2.2: Componentele URL-ului

Tipul de protocol URL Primul fragment din URL reprezintă tipul de protocol pe care browser-ul trebuie să îl folosească pentru a trimite o cerere de acces la resursă. Protocoalele cele mai întâlnite sunt HTTPS (Hypertext Transfer Protocol Secured) sau HTTP (varianta nesecurizată), dar există cazuri în care este folosit și protocolul mailto: care deschide un client de tip mail.



Figura 2.3: Tipul de protocol URL

Numele resursei și portul Numele resursei sau numele domeniului, separat de tipul de protocol prin tiparul de caractere `://`, reprezintă adresa paginii web redactată sub forma unui șir de caractere. Orice pagină web are atribuit un IP public unic din intervalul 1.0.0.0 - 9.255.255.255, numere care sunt greu de reținut sau scris pentru orice persoană. Din acest motiv, numele domeniului ajută în fluidizarea procesului de accesare a unei pagini web prin „traducerea în limbaj natural” a adreselor IP într-un format ușor de înțeles și accesibil pentru oricine. Portul este o „poartă” tehnică folosită pentru a accesa resursele pe server-ul web. În cele mai multe cazuri, declararea portului este omisă deoarece majoritatea serverelor folosesc în mod implicit portul 80 pentru HTTP și 443 pentru HTTPS, dar în situațiile în care este necesară utilizarea unui alt port, acesta trebuie declarat în mod obligatoriu.

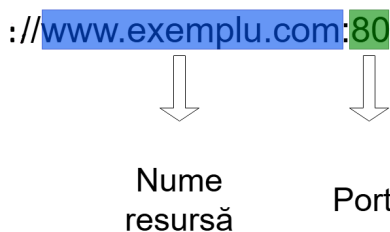


Figura 2.4: Numele resursei și portul

Calea către resursă Următoarea componentă a unui URL este calea către resursă. Calea indică locația exactă a unei pagini, imagini sau altei resurse aflate pe server.



Figura 2.5: Calea către resursă

Parametrii Parametrii sunt o listă de chei sau valori folosite de către server-ul web în a executa anumite operații înainte de a se ocupa de furnizarea resursei cerute. Pentru delimitarea fiecărui parametru se folosește simbolul &. Fiecare server web are o listă de chei sau de valori specifice, nu există o regulă specifică în crearea acestora.

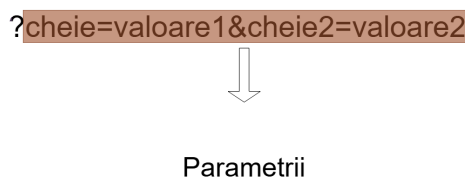


Figura 2.6: Parametrii

Fragment de identificare internă Un fragment de identificare internă sau ancora, delimitată de simbolul #, are rol în afișarea unei anumite secțiuni din resursă. Ancorele funcționează ca un fel de „semn de carte”, aceasta indicând aplicației web până unde va trebui să parcurgă documentul ca să ajungă la secțiunea marcată și într-un final să o afișeze.

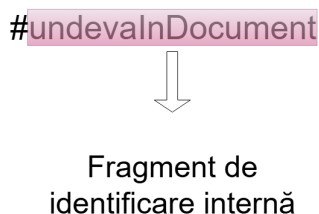


Figura 2.7: Ancoră URL

2.2.2 Conexiunea HTTP

Un server HTTP poate fi direct accesat prin numele domeniilor pe care site-urile web le dețin și livrează conținutul acestora către utilizatori. La nivel fundamental, modul de funcționare al unui server web presupune ca de fiecare dată când un browser are nevoie de un fișier găzduit de către un server web, acesta va trimite o cerere HTTP pentru a primi fișierul respectiv. Odată ce server-ul HTTP a recepționat cererea, verifică dacă URL solicitat există și în cazul confirmării, acesta trimite la utilizator conținutul cerut. În cazul în care server-ul nu reușește să verifice cererea, mai întâi acesta verifică dacă este necesară generarea unui fișier dinamic. În cele din urmă, dacă niciuna dintre opțiuni nu este disponibilă, server-ul web va returna codul de eroare 404 Not Found.

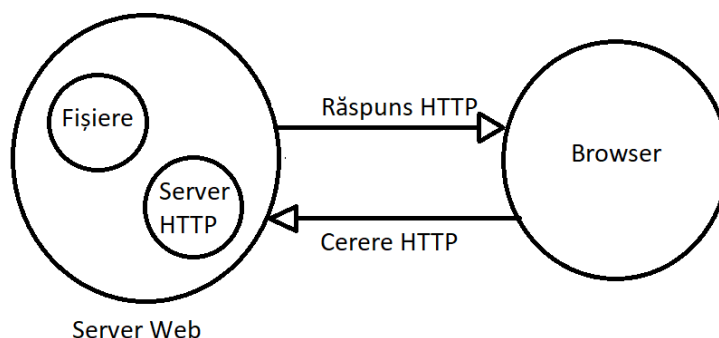


Figura 2.8: Diagrama modului de funcționare a server-ului web

2.2.2.1 Cum funcționează o conexiune HTTP

O conexiune HTTP este controlată la nivelul stratului de transport, ceea ce înseamnă că, de fapt, nu este rolul protocolului HTTP de a-și gestiona conexiunea, ci de a se folosi de un protocol de transport. Acesta are nevoie de un protocol de transport fiabil, prin care mesajele au o rată de pierdere foarte mică. Din acest motiv, se utilizează protocolul TCP datorită fiabilității sale care garantează să trimită mesajele cu un număr minim erori, deși acesta are o viteză de răspuns mai lentă în comparație cu protocolul UDP.

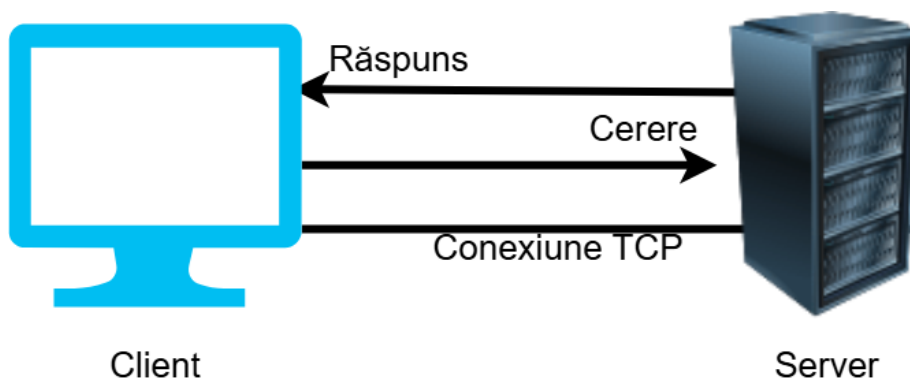


Figura 2.9: Diagrama conexiunii HTTP

Primul pas spre a începe o conexiune HTTP îl reprezintă schimbul de cerere/răspuns dintre client și server. Acest schimb începe odată ce s-a stabilit o conexiune TCP, un proces care trimite unul sau mai multe cereri din partea clientului pentru a primi un răspuns de la server. Clientul are posibilitatea de a deschide o nouă conexiune, de a folosi o conexiune existentă sau de a deschide mai multe conexiuni TCP către servere.

Al doilea pas presupune trimiterea mesajul HTTP către server și răspunsul server-ului la cererea clientului cu informația cerută. Dacă serverul răspunde cu codul 200 OK, înseamnă faptul că s-a stabilit cu succes conexiunea, iar serverul poate transmite clientului informația cerută.

```
HTTP
GET / HTTP/1.1
Host: localhost
Accept-Language: en
```

Figura 2.10: Cerere HTTP

```
HTTP
HTTP/1.1 200 OK
Date: Sat, 09 Oct 2025 14:28:02 GMT
Server: Apache
Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT
ETag: "51142bc1-7449-479b075b2891b"
Accept-Ranges: bytes
Content-Length: 29769
Content-Type: text/html

<!doctype html>... (conținutul paginii cerute)
```

Figura 2.11: Răspuns HTTP

În cele din urmă, conexiunea este închisă sau reutilizată.

2.2.3 Mesaje HTTP

Mesajele HTTP sunt șiruri de caractere scrise în limbaj natural, simplist, începând cu versiunea HTTP/1.1 până la versiunea HTTP/2. Reprezintă un mecanism de schimb de date între server și un client. Există două tipuri de mesaje: cerere și răspuns. Cererea este trimisă de către client server-ului pentru a declanșa o acțiune, iar răspunsul este informația trimisă de către server înapoi clientului. Modul prin care mesajele HTTP sunt create sau transformate se datorează API-urilor în browser, fișiere de configurație ale proxy-urilor sau serverelor sau altor interfețe.

2.2.3.1 Structura mesajelor HTTP

Conform figurei 2.12 se poate observa faptul că mesajele de tip cerere și răspuns prezintă o structură asemănătoare:

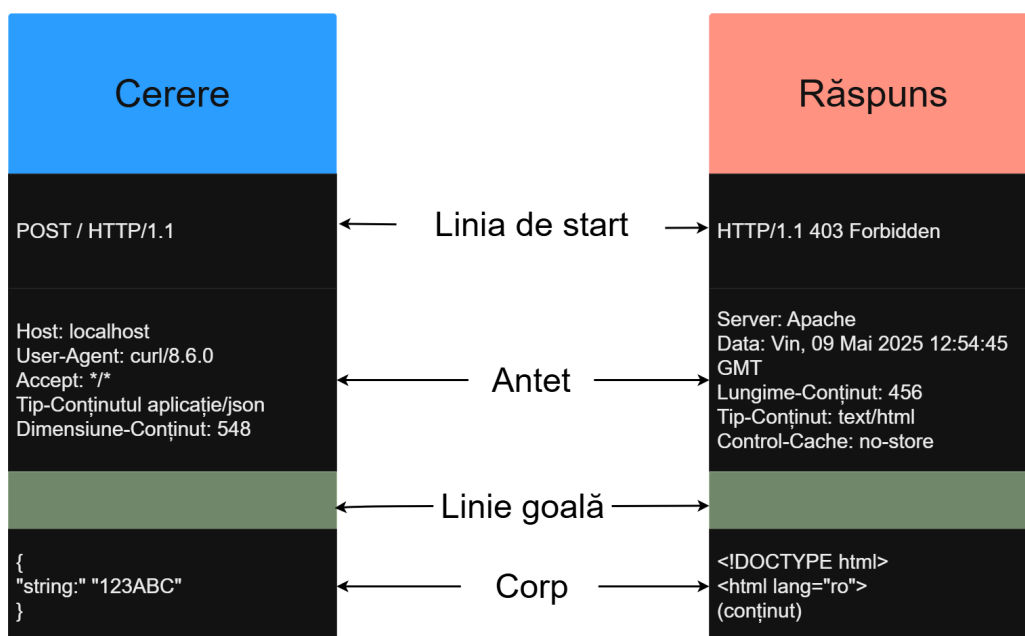


Figura 2.12: Răspuns HTTP

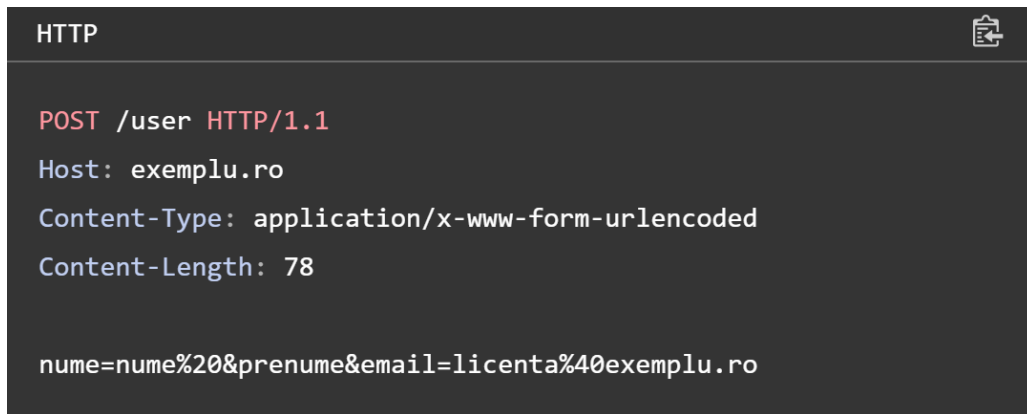
1. Linia de start este o singură linie care descrie versiunea HTTP folosită alături de metoda cererii sau răspunsul acesteia.
2. Setul opțional de antete HTTP conțin metadata care descriu mesajul transmis. Un antet HTTP permite schimbul de informații suplimentare dintre un client și un server în cadrul unei cereri sau răspuns. În funcție de context, se remarcă mai multe categorii de antet:
 - Antet de cerere (request header): conține informații despre resursa ce urmează a fi preluată.
 - Antet de răspuns (response header): conține informații despre răspuns cum ar fi locația sau serverul de pe care provine.
 - Antet de reprezentare (representation header): conține informații despre corpul resursei, cum ar fi tipul MIME sau tipul de compresie sau codare.

- Antet cu conținut util (payload header): conține informații de reprezentativ independente despre datele de conținut util
3. O linie goală care indică faptul că metadata mesajului este completă.
 4. Un corp (body) opțional care conține datele asociate mesajului, stabilit încă de la linia de start și din anteturi dacă va fi transmis alături de mesaj sau nu. De obicei conține o metodă trimisă de client către server sau o resursă redirectionată de către server clientului.

Partea superioară (head) a mesajului HTTP este compus din linia de start și antet, în timp ce corpul cuprinde restul componentelor.

2.2.3.2 Cereri HTTP

O cerere HTTP [11] este un mesaj trimis de client către server cu scop de a solicita o resursă. Aceasta este compusă dintr-o parte superioară ce cuprinde linia cererii (request line), antet (header), linie goală și corp.



```

HTTP

POST /user HTTP/1.1
Host: exemplu.ro
Content-Type: application/x-www-form-urlencoded
Content-Length: 78

nume=nume%20&prenume&email=licenta%40exemplu.ro

```

Figura 2.13: Cerere HTTP

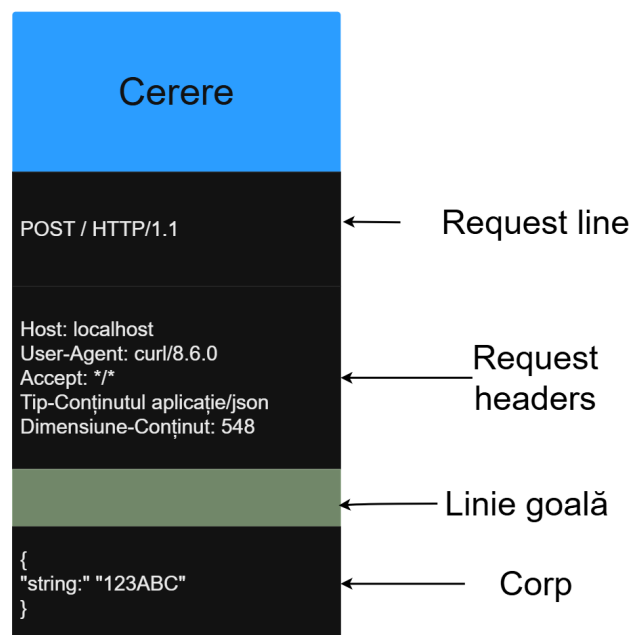


Figura 2.14: Cerere HTTP

Prima componentă a părții superioare a unei cereri HTTP o constituie linia cererii. Așa cum se observă și în figura 2.15, se pot observa trei părți ce compun linia cererii: metoda, resursa solicitată și protocolul.

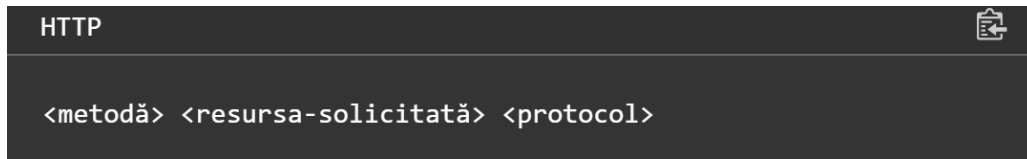


Figura 2.15: Formatul unei cereri HTTP

O metodă HTTP indică scopul unei cereri trimise către un server și rezultatul așteptat în cazul în care cererea a fost realizată cu succes. De obicei fiecare metodă are propriul său rol, dar pot împărtăși anumite caracteristici comune. Există mai multe tipuri de metode precum ar fi:

1. **GET** este folosită pentru doar a prelua date de pe server, fără ca acestea să sufere noi modificări.
2. **HEAD** se comportă în același mod ca și metoda GET, dar fără a include conținut în răspuns.
3. **POST** are rol în a trimite date către server, de exemplu trimiterea încărcarea unui fișier.
4. **PUT** înlocuiește toate instanțele curente cu noi instanțe specificate de către client.
5. **DELETE** este utilizată cu scopul de a solicita ștergerea unei resurse specificate.
6. **CONNECT** este utilizată pentru a crea un tunel de comunicație cu serverul identificat prin resursa țintă.
7. **OPTIONS** are rol în a descrie opțiunile de comunicare posibile pentru resursa specificată.
8. **TRACE** este folosită pentru efectuarea unui test de buclă (loop-back) a mesajului de-a lungul traseului către resursa specificată.
9. **PATCH** aplică modificări parțiale unei resurse.

Resursa solicitată este un URI (Uniform Resource Identifier) și identifică resursa căreia i se trimite cererea. Există mai multe tipuri de formatare utilizate pentru a descrie o resursă solicitată, cea mai folosită fiind forma de origine (origin form).

Forma de origine, utilizată cu metodele GET, POST, HEAD și OPTIONS, este folosită pentru a identifica originea resursei dintr-un server sau gateway. Presupune combinarea unei căi absolute cu informațiile aflate în antetul „Host” la care există posibilitatea adăugării unei interogări de forma *cheie* = *valoare* pentru a transmite informații suplimentare.

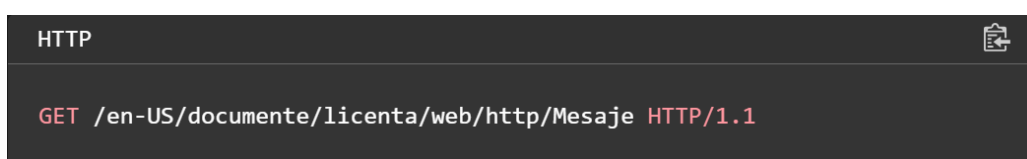


Figura 2.16: Forma de origine

Forma absolută este o sintaxă de tip URL complet, ce include informațiile necesare pentru identificarea serverului, în care cererea HTTP este făcută către un proxy, cu rol în redirectionarea răspunsului sau serviciul de la un cache valid înapoi către client, utilizând metoda GET.

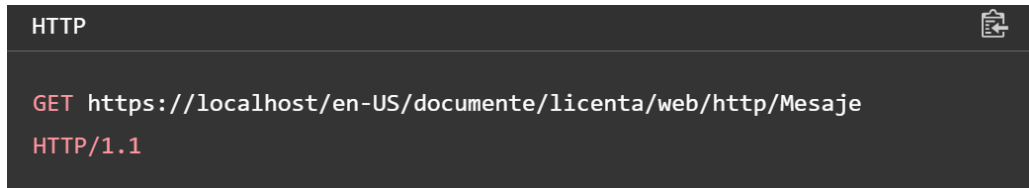
A screenshot of a dark-themed HTTP client interface. At the top, the word "HTTP" is displayed in white. Below it, the request line is shown in red text: "GET https://localhost/en-US/documente/licenta/web/http/Mesaje HTTP/1.1". A small icon of a document with a plus sign is visible in the top right corner of the interface.

Figura 2.17: Forma absolută

Forma de autoritate este o structură folosită în cererile HTTP formată dintr-un nume de gazdă și opțional un port delimitat de simbolul : împreună cu metoda CONNECTION pentru a seta un tunel către resursa țintă.

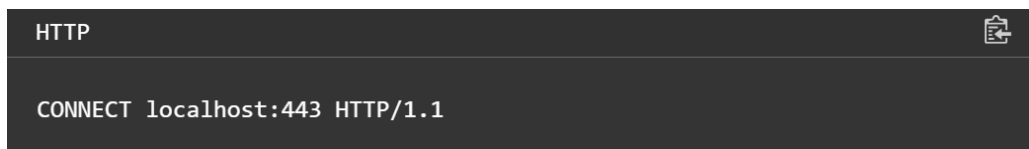
A screenshot of a dark-themed HTTP client interface. At the top, the word "HTTP" is displayed in white. Below it, the request line is shown in white text: "CONNECT localhost:443 HTTP/1.1". A small icon of a document with a plus sign is visible in the top right corner of the interface.

Figura 2.18: Forma de autoritate

Forma asterisk (*) este folosită alături de metoda OPTIONS atunci când cererea HTTP nu se aplică unei resurse particulare, ci server-ului ca un întreg.

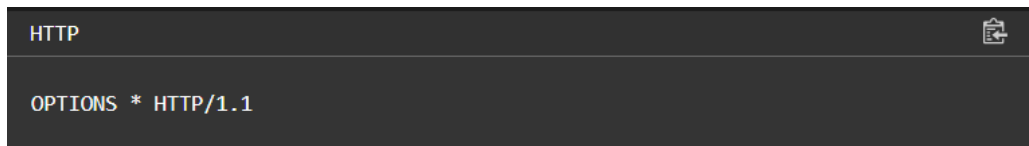
A screenshot of a dark-themed HTTP client interface. At the top, the word "HTTP" is displayed in white. Below it, the request line is shown in white text: "OPTIONS * HTTP/1.1". A small icon of a document with a plus sign is visible in the top right corner of the interface.

Figura 2.19: Forma asterisk

Ultima componentă a liniei de cerere se numește protocol. Este un indicator care reprezintă versiunea protocolului HTTP utilizată în timpul procesului de trimitere a cererii folosit pentru o interpretare corectă a mesajelor transmise. Singura versiune care trebuie declarată în acest câmp este HTTP/1.1, deoarece versiunile HTTP/0.9, HTTP/1.0 sunt învechite și nu se mai folosesc, iar versiunile mai noi de HTTP/2.0 sunt configurate de dinainte în timpul conexiunii.

Cea de-a doua componentă a părții superioare unei cereri HTTP o constituie antetul (header), care poate fi clasificat după rolul său în:

```
Host: localhost
User-Agent: curl/8.6.0
Accept: */*
Tip-Conținutul aplicație/json
Dimensiune-Conținut: 548
```

Figura 2.20: Header request

1. antet de cerere (request header) care furnizează context suplimentar în cadrul unui mesaj de tip cerere sau adaugă instrucțiuni despre cum ar trebui să fie un mesaj tratat de către server
2. antet de reprezentare (representation header), folosit numai în cazul în care cererea prezintă și o parte superioară, descrie forma originală a datelor din mesaj și forma de codare aplicată. Un representation request îi permite destinatarului să reconstruiască resursa așa cum era înainte de a fi transmisă în rețea. Câteva exemple de request header-uri des folosite într-o cerere sunt:

- Accept-Charset
- Host
- User-Agent
- Accept-Language
- Accept-Encoding
- Connection
- Referer

Ultima componentă a unei cereri este constituită din corpul mesajului care are rol în transmiterea informației către server, compatibilă doar cu metodele PATCH, POST și PUT. Corpul unui mesaj poate conține o cantitate scăzută de date sub formă de perechi de tip key=value (cheie=valoare) (2.21) sau date în mai multe părți (2.22):

```
JSON
{
  "nume": "Marin",
  "prenume": "Costel",
  "email": "lic@exemplu.com"
}
```

Figura 2.21: Date din corp sub formă de key=values



```

HTTP

--delimiter1
Content-Disposition: form-data; name="field0"

value1
--delimiter1
Content-Disposition: form-data; name="field1"; filename="exemplu.txt"

Text file contents
--delimiter1--

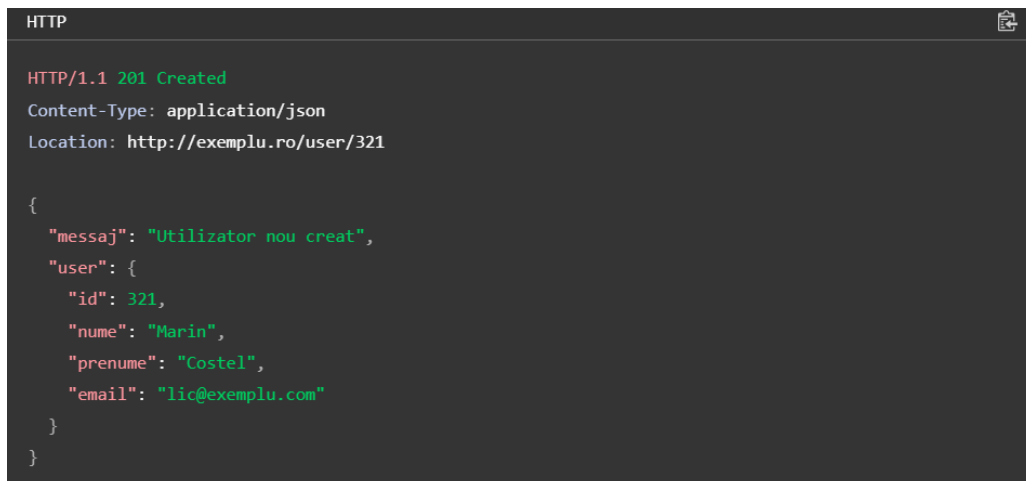
```

Figura 2.22: Date din corp împărțite în mai multe părți

2.2.3.3 Răspunsuri HTTP

Răspunsurile HTTP [12] sunt mesaje trimise de către server ca și răspuns înapoi către clienți. Răspunsul anunță clientul rezultatul determinat în urma cererii.

În figura 2.23 este descris un răspuns efectuat după realizarea operației POST, în care un nou utilizator este creat.



```

HTTP

HTTP/1.1 201 Created
Content-Type: application/json
Location: http://exemplu.ro/user/321

{
  "mesaj": "Utilizator nou creat",
  "user": {
    "id": 321,
    "nume": "Marin",
    "prenume": "Costel",
    "email": "lic@exemplu.com"
  }
}

```

Figura 2.23: Răspuns HTTP

Așa cum se poate observa din figura anterioară, un răspuns este alcătuit din trei componente: status line, response headers și corpul mesajului.

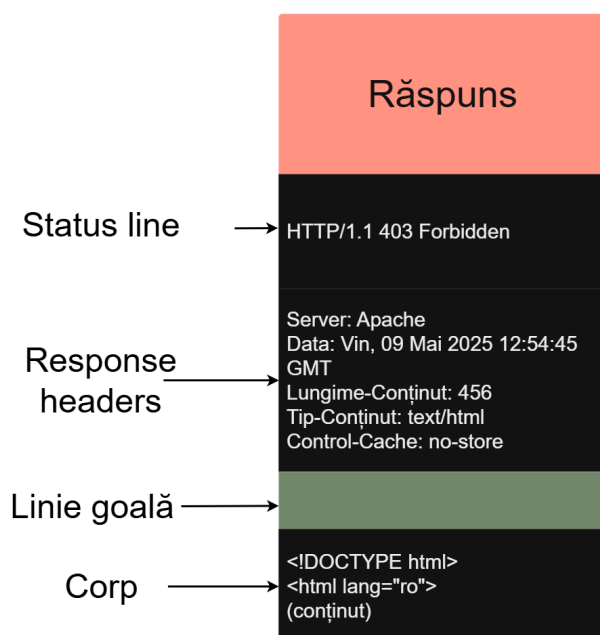


Figura 2.24: Alcătuirea răspunsului HTTP

Status line reprezintă prima linie a părții superioare al unui răspuns HTTP. Acesta este format din trei componente:

1. Protocolul descrie versiunea protocolului HTTP pe care l-a folosit serverul pentru a transmite mesajul către client.
2. Cod de stare indică dacă un răspuns HTTP a fost completat cu succes. Există cinci tipuri de coduri de stare cum ar fi: răspunsuri informaționale (cu valori între 100 și 199), răspunsuri ce descriu transmiterea cu succes (200-299), mesaje de redirecționare (300-399), răspunsuri pentru erori de client (400-499) și răspunsuri pentru erori de server (500-599). Cele mai des întâlnite coduri de stare sunt 200 - OK (mesajul a fost transmis cu succes), 404 - Not Found (serverul nu poate găsi resursa cerută, URL nu este recunoscut de către browser), 302 - Found (URL-ul asociat resursei căutate a fost schimbat temporar).
3. Textul de stare este un text pur informațional ce detaliază codul de stare în limbaj natural pentru o mai bună înțelegere a statusului situației.



Figura 2.25: Status line - alcătuire

Cea de-a doua componentă a părții superioare al unui răspuns este alcătuită din response header și representation header.

Un response header este un antet cu metadatele trimise odată cu răspunsul de server către client. Fiecare antet este un șir de caractere care nu face diferențe între majuscule și litere mici, urmate de simbolul : și o valoare care este dependentă de tipul de antet folosit. Scopul

response header-ului este de a oferi informații suplimentare despre mesaj sau de a adăuga logica despre cum ar trebui clientul să gestioneze cererile ce o să urmeze. Representation header-ul descrie forma datelor din mesaj și orice codare aplicată asupra acestuia și poate exista doar dacă mesajul are un corp. Un representation header oferă unui destinatar instrucțiuni despre cum ar trebui să reconstruiască resursa înainte de a fi trimisă în rețea.

```
Server: Apache
Data: Vin, 09 Mai 2025 12:54:45
GMT
Lungime-Conținut: 456
Tip-Conținut: text/html
Control-Cache: no-store
```

Figura 2.26: Exemplu de response și representation header

Ultima componentă a unui răspuns este alcătuită dintr-un corp, inclusiv în marea majoritatea a mesajelor atunci când un server trimite un răspuns către client. În răspunsurile realizate cu succes, corpul mesajului conține datele pe care un client le-a cerut prin intermediul metode GET. În schimb, dacă apar erori în transmiterea mesajului, se vor detalia problemele care au cauzat imposibilitatea trimiterii răspunsului în corp.

2.3 Flask Python

Flask [13] [14] este un framework flexibil web minimalist creat în limbajul Python, deseori numit „micro-framework”. Datorită accesibilității de a dezvolta aplicații web folosind doar componente esențiale, fără necesitatea folosirii unor biblioteci suplimentare sau structuri complicate, dezvoltatorii au oportunitatea de a implementa și adăuga ușor funcționalități în aplicațiile web.

Flask a fost creat [15] în anul 2004 de către Armin Ronacher, un membru al grupului de pasionați de Python, Poccoo. Inițial, crearea celebrului framework a fost doar o aluzie la celebra zi a glumelor 1 aprilie, dar care mai târziu a devenit o situație serioasă. Numele „flask” este o referință la precedentul framework Bottle. În anul 2016, echipa Poccoo a fost desființată, iar dezvoltarea Flask-ului și a librăriilor acestuia a fost preluată de către echipa nou fondată Pallets.

În contextul acestei lucrări de licență, am ales să implementez serverul aplicației web în Flask datorită de avantaje pe care framework-ul le aduce cum ar fi:

- Este un framework backend minimalist cu posibilitatea extinderii aplicației în funcție de cerințele impuse.
- Este ușor de învățat deoarece API-ul intuitiv oferă posibilitatea chiar și începătorilor să se familiarizeze cu el.
- Este un framework flexibil deoarece are abilitatea de a permite personalizarea și extinderea aplicației conform cerințelor cerute.
- Poate fi folosit cu o paletă largă de baze de date precum MySQL prin intermediul extensiilor sau al bibliotecilor externe.
- Routing HTTP care se ocupă cu definirea rutelor URL și asocierea acestora cu funcții care gestionează cererile și răspunsurile HTTP.

- Sesiunile de utilizatori pot fi ușor gestionate, deoarece Flask are posibilitatea de a stoca informații despre acestea în mod securizat pe server și client utilizând cookie-uri semnate.
- Posibilă integrarea cu Jinja2 care oferă o paletă largă de tool-uri pentru generarea paginilor HTML dinamice.

Capitolul 3

Structura aplicației

Aplicația a fost structurată în șapte module, fiecare modul având un rol bine definit în arhitectura generală. Acestea sunt următoarele:

1. Modulul de aplicație web în Python Flask este prezent pe stația de lucru și include rute importante precum `/`, `/login`, `/register`, `/logout`, `/profile`, `/admin`, `/reserve` pentru gestionarea logicii unui sistem de management al rezervărilor, autentificarea utilizatorilor și interacțiunea cu baza de date. Funcționalitățile rutelor sunt:
 - `/login` - autentificarea utilizatorilor și validarea credențialelor.
 - `/register` - înregistrarea utilizatorilor noi și validarea datelor.
 - `/logout` - finalizarea sesiunii active.
 - `/reserve` - rezervarea unui hotel (operațiune de tip CRUD).
 - `/profile` - afișarea și actualizarea profilelor utilizatorilor.
 - `/admin` - rută pentru utilizatorii cu rol de administrator.
 - `/` - pagina de acasă cu o interfață generală sau redirecționare în funcție de sesiune.
2. Modulul de load balancing (Nginx): Nginx (instalat pe stația de lucru) se ocupă de distribuirea cererilor primite de la utilizatori către cele două servere: aplicația locală Flask și serverul IIS de pe VM2, ceea ce îmbunătățește scalabilitatea soluției implementate. Load balancer-ul are mai multe roluri cum ar fi:
 - Reverse proxy care asigură redirecționarea traficului.
 - Verificarea și detectarea automată a sănătății server-ului și scoaterea temporară din funcționare până când sunt rezolvate erorile.
 - Balansarea cererilor trimise către servere, astfel încât cereri sunt preluate secvențial.
 - Măsură de rezervă în cazul în care unul dintre servere devine inaccesibil, traficul va fi redirecționat către următorul server disponibil.
3. Modulul de testare are rol în testarea izolată a aplicației web fără ca sistemul principal să fie afectat. Pentru acest mediu de testare izolat s-a folosit VM1 care are responsabilitatea de a testa codul Flask, conectivitatea cu serviciul MySQL, validarea configurației Nginx, rularea unui debug server Flask și de utilizarea a aplicației Postman sau chiar a browser-ului local pentru a rula testele.
4. Modulul de server IIS reprezintă o a doua instanță web care poate fi folosit pentru a replica parțial funcționalități ale aplicației Flask, distribuirea paginilor, resurselor statice (HTML, JavaScript, CSS), resurselor REST și pentru a răspunde la cererile distribuite de către load balancer.

5. Modulul de bază de date MySQL Master, ce se află pe stația de lucru, are la bază nodul principal al bazei de date unde toate operațiile de scriere (INSERT, UPDATE, DELETE) se fac pe acest server. Principalele obiective ale modulului sunt de a gestiona tabelele aplicației web, replicarea automată către slave (VM2), creare utilizatorilor și a privilegiilor pentru replicare și acceptarea conexiunilor de la aplicația Flask.
6. Modulul de bază de date MySQL Slave are rolul de a prelua și stoca copii ale datelor scrise pe master în timp real pentru a efectua operații de citire (SELECT) asupra bazei de date. Această tehnică asigură îmbunătățirea performanței aplicației atunci când este folosită frecvent comanda SELECT și asigură redundanța datelor. Un alt rol crucial pe care îl reprezintă slave este acela că poate fi promovat la master în cazul în care nodul principal eșuează, astfel datele nu vor putea fi pierdute sau compromise.
7. Modulul de rețea și virtualizare asigură comunicarea între stația de lucru, VM1 și VM2 prin intermediul rețelei bridged sau NAT în VMware Workstation. La nivelul fiecărei mașini sunt setate IP-uri statice pentru a asigura o conexiune stabilă.

Capitolul 4

Noțiuni teoretice privind aplicațiile web și scalabilitatea

Pentru facilitarea formatării tezei de licență/dizertație în $\text{\LaTeX}$, se furnizează fișierul clasă “upb_thesis.cls” și un model de folosire “exemplu_teza.tex”. Utilizarea comenzilor standard $\backslash\text{documentclass}$, usepackage , setcounter , etc. nu este diferită de utilizarea lor în orice alt context $\text{\LaTeX}$. Comenzile standard sau cele specifice UPB apar într-o secvență care trebuie respectată pentru a obține documentul în formatul acceptat de universitate. Pentru redactarea tezei, este obligatorie folosirea următoarelor comenzi:

$\backslash\text{singlespacing}$ specifică spațierea pentru întregul document.

$\backslash\text{author}\{\text{Ion Creangă}\}$ numele studentului va fi inserat pe pagina de titlu și în declarația de onestitate.

$\backslash\text{title}\{\text{Basmelor românilor}\}$ titlul tezei

$\backslash\text{facultatea}\{\text{Facultatea de Electronică, Telecomunicații și Tehnologia Informației}\}$

$\backslash\text{tiplucrare}\{\text{licență}\}$ parametrul trebuie să fie “licență” sau “dizertație”

$\backslash\text{domeniu}\{\text{Electronică, Telecomunicații și Tehnologia Informației}\}$

$\backslash\text{catedra}\{\}$ Conform anexei 4 din ghidula absolventului.

$\backslash\text{program}\{\text{Povești}\}$ numele specializării

$\backslash\text{titlulobtinut}\{\text{Inginer}\}$ parametrul trebuie să fie “Inginer” sau “Master”

$\backslash\text{director}\{\text{Mihail Kogălniceanu}\}$ numele îndrumătorului de proiect. Dacă sunt mai mulți, se vor separa cu doi backslash și un spațiu:

$\backslash\text{director}\{\text{Mihail Kogălniceanu}\backslash\backslash\text{Titu Maiorescu}\}$

$\backslash\text{submissionmonth}\{\text{Iunie}\}$

$\backslash\text{submissionyear}\{\text{2011}\}$ data care apare pe pagina de titlu

Listele opționale de figuri, tabele, și abrevieri trebuie incluse în secvența

$\backslash\text{beforepreface}$ - $\backslash\text{afterpreface}$:

$\backslash\text{beforepreface}$

$\backslash\text{listoffigures}$

$\backslash\text{listoftables}$

$\backslash\text{abbreviations}\{$

$\}$

$\backslash\text{afterpreface}$

4.1 Alte recomandări

- Se recomandă folosirea pachetului aspell (pachetul `aspell-ro` în Debian/Ubuntu) pentru verificarea ortografiei, cu comanda:

```
aspell --lang=ro -t check ./exemplu_teza.tex
```

- Se recomandă verificarea fonturilor în PDF-ul produs - este preferabil să nu se folosească decât “Type 1” sau “Truetype”, nu “Type 3”.

Capitolul 5

Basme

5.1 Punguța cu doi bani

Era odată o babă și un moșneag. Baba avea o găină, și moșneagul un cucuș; găina babei se oua de câte două ori pe fiecare zi și baba mânca o mulțime de ouă; iar moșneagului nu-i da nici unul. Moșneagul într-o zi perdu răbdarea și zise:

— Măi babă, mănânci ca în târgul lui Cremene. Ia dă-mi și mie niște ouă, ca să-mi prind pofta măcar. — Da' cum nu! zise baba, care era foarte zgârcită. Dacă ai poftă de ouă, bate și tu cucușul tău, să facă ouă, și-i mânca; că eu așa am bătut găina, și iacătă-o cum se ouă.

Moșneagul, pofticios și hapsin, se ia după gura babei și, de ciudă, prinde iute și degrabă cucușul și-i dă o bataie bună, zicând:

— Na! ori te ouă, ori du-te de la casa mea; ca să nu mai strici mâncarea degeaba.

Cucușul, cum scapă din mâinile moșneagului, fugi de-acasă și umbla pe drumuri, bezmetec. Și cum mergea el pe-un drum, numai iată găsește o punguță cu doi bani (Figura 5.1). Și cum o găsește, o și ia în clonț și se întoarce cu dânsa înapoi către casa moșneagului. Pe drum se întâlnește c-o trăsură c-un boier și cu niște cucoane. Boierul se uită cu băgare de seamă la cucuș, vede în clonțu-i o punguță și zice vezeteului:

— Măi! ia dă-te jos și vezi ce are cucușul cela în plisc.

Vezeteul se dă iute jos din capra trăsorei, și c-un feliu de meșteșug, prinde cucușul și luându-i punguța din clonț o dă boieriului. Boieriul o ia, fără pasare o pune în buzunar și pornește cu trăsura înainte. Cucușul, supărat de asta, nu se lasă, ci se ia după trăsura, spuind neîncetat:

Cucurigu ! boieri mari, Dați punguța cu doi bani !

[...]

(Publicată pentru prima oară în [16].)

5.2 Tinerețe fără bătrânețe și viață fără de moarte

A fost odată ca niciodată; că de n-ar fi, nu s-ar mai povesti; de când făcea ploșorul pere și răchita micșunele; de când se băteau urșii în coade; de când se luau de gât lupii cu mieii de se sărutau, înfrățindu-se; de când se potcovea puricele la un picior cu nouăzeci și nouă de oca de fier și s-arunca în slava cerului de ne aducea povești;

De când se scria musca pe părete, Mai mincinos cine nu crede.

A fost odată un împărat mare și o împărăteasă, amândoi tineri și frumoși, și, voind să aibă copii, a făcut de mai multe ori tot ce trebuia să facă pentru aceasta; a îmblat pe la vraci[17] și filosofi, ca să caute la stele și să le ghicească dacă or să facă copii; dar în zadar. În sfârșit, auzind împăratul că este la un sat, aproape, un unchiaș dibaci, a trimis să-l cheme; dar el răspunse trimișilor că: cine are trebuință, să vie la dânsul. S-au sculat deci împăratul și împărăteasa și, luând cu dâșii vro câțiva boieri mari, ostași și slujitori, s-au dus la unchiaș acasă. Unchiașul, cum i-a văzut de departe, a ieșit să-i întâmpine și totodată le-a zis:

- Bine ați venit sănătoși; dar ce îmblî, împărate, să afli? Dorința ce ai o să-ți aducă întristare.

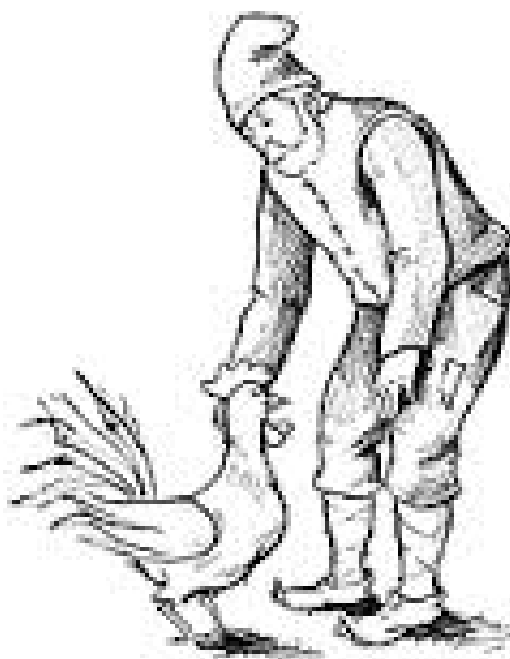


Figura 5.1: Cucoșu găsește o pungă cu doi galbeni.

- Eu nu am venit să te întreb asta, zise împăratul, ci, dacă ai ceva leacuri care să ne facă să avem copii, să-mi dai.

- Am, răspunse unchiașul; dar numai un copil o să faceți. El o să fie Făt-Frumos și dragăstos, și parte n-o să aveți de el. Luând împăratul și împărăteasa leacurile, s-au întors veseli la palat și peste câteva zile împărăteasa s-a simțit însărcinată. Toată împărăția și toată curtea și toți slujitorii s-au veselit de această întâmplare.

Mai-nainte de a veni ceasul nașterii, copilul se puse pe un plâns, de n-a putut nici un vraci să-l împace. Atunci împăratul a început să-i făgăduiască toate bunurile din lume, dar nici așa n-a fost cu putință să-l facă să tacă.

- Taci, dragul tatei, zice împăratul, că ți-oi da împărăția cutare sau cutare; taci, fiule, că ți-oi da soție pe cutare sau cutare fată de împărat, și alte multe d-alde astea; în sfârșit, dacă văzu și văzu că nu tace, îi mai zise: taci, fătul meu, că ți-oi da Tinerețe fără bătrânețe și viață fără de moarte.

Atunci, copilul tăcu și se născu; iar slujitorii deteră în timpine și în surle și în toată împărăția se ținu veselie mare o săptămână întreagă.

De ce creștea copilul, d-aceea se făcea mai istet și mai îndrăzneț. Îl deteră pe la școli și filosofi, și toate învățăturile pe care alți copii le învăța într-un an, el le învăța într-o lună, astfel încât împăratul murea și învia de bucurie. Toată împărăția se fălea că o să aibă un împărat înțelept și procopsit ca Solomon împărat. De la o vreme încoace însă, nu știu ce avea, că era tot galeș, trist și dus pe gânduri. Iar când fuse într-o zi, tocmai când copilul împlinea cincisprezece ani și împăratul se afla la masă cu toți boierii și slujbașii împărăției și se chefuiau, se sculă Făt-Frumos și zise:

- Tată, a venit vremea să-mi dai ceea ce mi-ai făgăduit la naștere.

Auzind aceasta, împăratul s-a întristat foarte și i-a zis:

- Dar bine, fiule, de unde pot eu să-ți dau un astfel de lucru nemaiauzit? Și dacă ți-am făgăduit atunci, a fost numai ca să te împac.

- Dacă tu, tată, nu poți să-mi dai, apoi sunt nevoit să cutreier toată lumea până ce voi găsi făgăduința pentru care m-am născut.

Atunci toți boierii și împăratul deteră în genuchi, cu rugăciune să nu părăsească împărăția;

fiindcă, ziceau boierii:

— Tatăl tău de aci înainte e bătrân, și o să te ridicăm pe tine în scaun, și avem să-ți aducem cea mai frumoasă împărăteasă de sub soare de soție.

Dar n-a fost putință să-l întoarcă din hotărârea sa, rămânând statornic ca o piatră în vorbele lui; iar tată-său, dacă văzu și văzu, îi dete voie și puse la cale să-i gătească de drum merinde și tot ce-i trebuia.

[...]

5.3 Cinci pâini ¹

Doi oameni, cunoscuți unul cu altul, călătoreau odată, vara, pe un drum. Unul avea în traista sa trei pâni, și celalalt două pâni. De la o vreme, fiindu-le foame, poposesc la umbra unei răchiți pletoase, lângă o fântână cu ciutură, scoate fiecare pânilor ce avea și se pun să mănânce împreună, ca să aibă mai mare poftă de mâncare.

Tocmai când scoaseră pânilor din traiste, iaca un al treilea drumeț, necunoscut, îi ajunge din urmă și se oprește lângă dânsii, dându-le ziua bună. Apoi se roagă să-i deie și lui ceva de mâncare, căci e tare flămând și n-are nimica merinde la dânsul, nici de unde cumpăra.

— Poftim, om bun, de-i ospăta împreună cu noi, ziseră cei doi drumeți călătorului străin; căci mila Domnului! unde mănâncă doi mai poate mânca și al treilea.

Călătorul străin, flămând cum era, nemaiașteptând multă poftire, se așază jos lângă cei doi, și încep a mânca cu toții pâne goală și a be apă rece din fântână, căci altă udătură nu aveau. Și mănâncă ei la un loc tustrei, și mănâncă, până ce gătesc de mâncat toate cele cinci pâni, de parcă n-au mai fost.

După ce-au mântuit de mâncat, călătorul străin scoate cinci lei din pungă și-i dă, din întâmplare, celui ce avusese trei pâni, zicând:

— Primiți, vă rog, oameni buni, această mică mulțămărită de la mine, pentru că mi-ați dat demâncare la nevoie; veți cinsti mai încolo câte un pahar de vin, sau veți face cu banii ce veți pofti. Nu sunt vrednic să vă mulțămesc de binele ce mi-ați făcut, căci nu vedeam lumea înaintea ochilor de flămând ce eram.

Cei doi nu prea voiau să primească, dar, după multă stăruință din partea celui al treilea, au primit. De la o vreme, călătorul străin și-a luat ziua bună de la cei doi și apoi și-a căutat de drum. Celalți mai rămân oleacă sub răchită, la umbră, să odihnească bucatele. Și, din vorbă în vorbă, cel ce avuse trei pâni dă doi lei celui cu două pâni, zicând:

— Ține, frate, partea dumată, și fă ce vrei cu dânsa. Ai avut două pâni întregi, doi lei ți se cuvin. Și mie îmi opresc trei lei, fiindcă-am avut trei pâni întregi, și tot ca ale tale de mari, după cum știi.

— Cum așa?! zise celălalt cu dispreț! pentru ce numai doi lei, și nu doi și jumătate, partea dreaptă ce ni se cuvine fiecăruia? Omul putea să nu ne deie nimic, și atunci cum rămânea?

— Cum să rămâie? zise cel cu trei pâni; atunci așa fi avut eu pomană pentru partea ce mi se cuvine de la trei pâni, iar tu, de la două, și pace bună. Acum, însă, noi am mâncat degeaba, și banii pentru pâne îi avem în pungă cu prisos: eu trei lei și tu doi lei, fiecare după numărul pânilor ce am avut. Mai dreaptă împărțea decât aceasta nu cred că se mai poate nici la Dumnezeu sfântul...

— Ba nu, prietene, zice cel cu două pâni. Eu nu mă țin că mi-ai făcut parte dreaptă. Haide să ne judecăm, și cum a zice judecata, așa să rămâie.

¹Anecdota publicată prima oară în *Convorbiri literare*, nr. 12, 1 martie 1883

— Haide și la judecată, zise celălalt, dacă nu te mulțamești. Cred că și judecata are să-mi găsească dreptate, deși nu m-am târât prin judecăți de când sunt.

Și așa, pornesc ei la drum, cu hotărârea să se judece. Și cum ajung într-un loc unde era judecătorie, se înfățișează înaintea judecătorului și încep a spune împrejurarea din capăt, pe rând fiecare; cum a venit întâmplarea de au călătorit împreună, de au stat la masă împreună, câte pâni a avut fiecare, cum a mâncat drumețul cel străin la masa lor, deopotrivă cu dâșii, cum le-a dat cinci lei drept mulțămită și cum cel cu trei pâni a găsit cu cale să-i împartă.

Judecătorul, după ce-i ascultă pe amândoi cu luare aminte, zise celui cu două pâni: — Și nu ești mulțămit cu împărțeala ce s-a făcut, omule?

— Nu, domnule judecător, zise nemulțămitul; noi n-am avut de gând să luăm plată de la drumețul străin pentru mâncarea ce i-am dat; dar, dac-a venit întâmplarea de-așa, apoi trebuie să împărțim drept în două ceea ce ne-a dăruit oaspetele nostru. Așa cred eu că ar fi cu cale, când e vorba de dreptate.

— Dacă e vorba de dreptate, zise judecătorul, apoi fă bine de înapoieste un leu istualalt, care spui c-a avut trei pâni.

— De asta chiar mă cuprinde mirare, domnule judecător, zise nemulțămitul cu îndrăzneală. Eu am venit înaintea judecăței să capăt dreptate, și văd că dumneata, care știi legile, mai rău mă acufunzi. De-a fi să fie tot așa și judecata dinaintea lui Dumnezeu, apoi vai de lume!

— Așa ți se pare dumitale, zise judecătorul liniștit, dar ia să vezi că nu-i așa. Ai avut dumneata două pâni?

— Da, domnule judecător, două am avut.

— Tovarășul dumitale, avut-a trei pâni?

— Da, domnule judecător, trei a avut.

— Udătură ceva avut-ați vreunul?

— Nimic, domnule judecător, numai pâne goală și apă răce din fântână, fie de sufletul cui a făcut-o acolo, în calea trecătorilor.

— Dinioarea, parcă singur mi-ai spus, zise judecătorul, că ați mâncat toți tot ca unul de mult; așa este?

— Așa este domnule judecător.

— Acum, ia să statornicim rânduiala următoare, ca să se poată ști hotărât care câtă pâne a mâncat. Să zicem că s-a tăiat fiecare pâne în câte trei bucăți deopotrivă de mari; câte bucăți ai fi avut dumneata, care spui că avuși două pâni?

— Șese bucăți aș fi avut, domnule judecător.

— Dar tovarășul dumitale, care spui că avu trei pâni?

— Nouă bucăți ar fi avut, domnule judecător.

— Acum, câte fac la un loc șese bucăți și cu nouă bucăți?

— Cincisprezece bucăți, domnule judecător.

— Câți oameni ați mâncat aceste cincisprezece bucăți de pâne?

— Trei oameni, domnule judecător.

— Bun! Câte câte bucăți vin de fiecare om?

— Câte cinci bucăți, domnule judecător.

— Acum, ții minte câte bucăți ai fi avut dumneta?

— Șese bucăți, domnule judecător.

— Dar de mâncat, câte ai mâncat dumneta?

— Cinci bucăți, domnule judecător.

— Și câte ți-au mai rămas de întrecut?

— Numai o bucată, domnule judecător.

— Acum să stăm aici, în ceea ce te privește pe dumneta, și să luăm pe istalalt la rând. Ții minte câte bucăți de pâne ar fi avut tovarășul d-tale?

— Nouă bucăți, domnule judecător.

— Și câte a mâncat el de toate?

— Cinci bucăți, ca și mine, domnule judecător.

— Dar de întrecut, câte i-au mai rămas?

— Patru bucăți, domnule judecător.

— Bun! Ia, acuș avem să ne înțelegem cât se poate de bine! Vra să zică, dumneta ai avut numai o bucată de întrecut, iar tovarășul dumitale, patru bucăți. Acum, o bucată de pâne rămasă de la dumneta și cu patru bucăți de la istalalt fac la un loc cinci bucăți?

— Taman cinci, domnule judecător.

— Este adevărat că aceste bucăți de pâne le-a mâncat oaspetele dumneavoastră, care spui că v-a dat cinci lei drept mulțămîtă?

— Adevărat este, domnule judecător.

— Așadar, dumitale ți se cuvine numai un leu, fiindcă numai o bucată de pâne ai avut de întrecut, și aceasta ca și cum ai fi avut-o de vânzare, deoarece ați primit bani de la oaspetele dumneavoastră. Iar tovarășul dumitale i se cuvin patru lei, fiindcă patru bucăți de pâne a avut de întrecut. Acum, dară, fă bine de înapoieste un leu tovarășului dumitale. Și dacă te crezi nedreptățit, du-te și la Dumnezeu, și las' dacă ți-a face și el judecată mai dreaptă decât aceasta!

Cel cu două pâni, văzând că nu mai are încotro șovăi, înapoieste un leu tovarășului său, cam cu părere de rău, și pleacă rușinat. Cel cu trei pâni însă, uimit de așa judecată, mulțamește judecătorului și apoi iese, zicând cu mirare:

— Dac-ar fi pretutindene tot asemenea judecători, ce nu iubesc a li cânta cucul din față, cei ce n-au dreptate n-ar mai năzui în veci și-n pururea la judecată.

Corciogarii, porecliți și apărători, nemaiavând chip de traiu numai din minciuni, sau s-ar apuca de muncă, sau ar trebui, în toată viața lor, să tragă pe dracul de coadă... Iar societatea bună ar rămâne nebântuită.

Bibliografie

- [1] Adam Hayes. Scalability: What a scalable company is and examples. <https://www.investopedia.com/terms/s/scalability.asp>. Accesat: 08 mai 2025.
- [2] Kirill Stashevsky Nadejda Alkhaldi. What is software scalability, and why should your company take it seriously? <https://itrexgroup.com/blog/what-is-software-scalability>. Accesat: 08 mai 2025.
- [3] LinkedIn Computer Engineering. What is the best way to ensure hardware scalability in a computer system? <https://www.linkedin.com/advice/1/what-best-way-ensure-hardware-scalability-2pf1e>. Accesat: 08 mai 2025.
- [4] Wikipedia. Bază de date distribuită. https://ro.wikipedia.org/wiki/Baz%C4%83_de_date_distribuit%C4%83. Accesat: 08 mai 2025.
- [5] Yuliia Podorozhko. Scalability in software development: A 101 guide for businesses. <https://madappgang.com/blog/scalability-in-software-development/#:~:text=Scalability%20in%20software%20development%20refers,users%2C%20customers%2C%20or%20requests>. Accesat: 09 mai 2025.
- [6] Silviu Stroe. Ce este o aplicație web? <https://brainic.io/ro/blog/ce-este-o-aplicatie-web/>. Accesat: 09 mai 2025.
- [7] Developer Mozilla. Http documentation. <https://developer.mozilla.org/en-US/docs/Web/HTTP>. Accesat: 09 mai 2025.
- [8] Developer Mozilla. Http documentation. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/User-Agent>. Accesat: 09 mai 2025.
- [9] Developer Mozilla. Http documentation. https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_web_server. Accesat: 09 mai 2025.
- [10] Developer Mozilla. Http documentation. https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_URL. Accesat: 09 mai 2025.
- [11] Tutorialspoint. Http - requests. https://www.tutorialspoint.com/http/http_requests.htm. Accesat: 10 mai 2025.
- [12] Tutorialspoint. Http - responses. https://www.tutorialspoint.com/http/http_responses.htm. Accesat: 12 mai 2025.
- [13] Flask Tutorial. Geeksforgeeks. <https://www.geeksforgeeks.org/flask-tutorial/>. Accesat: 12 mai 2025.
- [14] Flask Documentation. Flask. <https://flask.palletsprojects.com/en/stable/>. Accesat: 12 mai 2025.

- [15] Wikipedia. Flask (web framework). [https://en.wikipedia.org/wiki/Flask_\(web_framework\)](https://en.wikipedia.org/wiki/Flask_(web_framework)). Accesat: 12 mai 2025.
- [16] Ion Creangă. Punguța cu doi bani. In *Convorbiri Literare*, number 10, ianuarie 1876.
- [17] Andrew Tanenbaum. *Computer Networks*. Prentice Hall Professional Technical Reference, 4th edition, 2002.

Anexa A

Rezolvare ”Cinci Pâini”

A aduce 3 pâini, iar B aduce 2 pâini. C plătește 5 lei. Se împarte fiecare pâine în 3 părți egale: $3 \times 3 + 2 \times 3 = 15$ bucăți. Fiecare mesean capătă câte 5 bucăți. Reiese că A i-a dat lui C $3 \times 3 - 5 = 4$ pâini, iar B i-a dat $2 \times 3 - 5 = 1$ pâine. Împărțeală cinstită este deci 4 lei pentru A, și 1 leu pentru B.

Anexa B

Alte povești și basme

Vezi tabela B cu alte povești și autorii/culegătorii lor.

```

1 function [ out ] = Cod( varinput )
2 % acesta este un comentariu
3 % param intrare varinput
4 % param iesire out
5
6 rez = 0;
7 n=10;
8 for i=1:n
9     for j=i+1:n-1
10        if abs(-2)<abs(-3)
11            rez = rez +1;
12        end
13    end
14 end
15 end

```

-	Dănilă prepeleac	Prâslea cel vo- nic și merele de aur	Făt-Frumos din lacrimă	Greuceanu
	Ion Creangă	Petre Ispirescu	Mihai Eminescu	Petre Ispirescu

Tabela B.1: Alte basme.